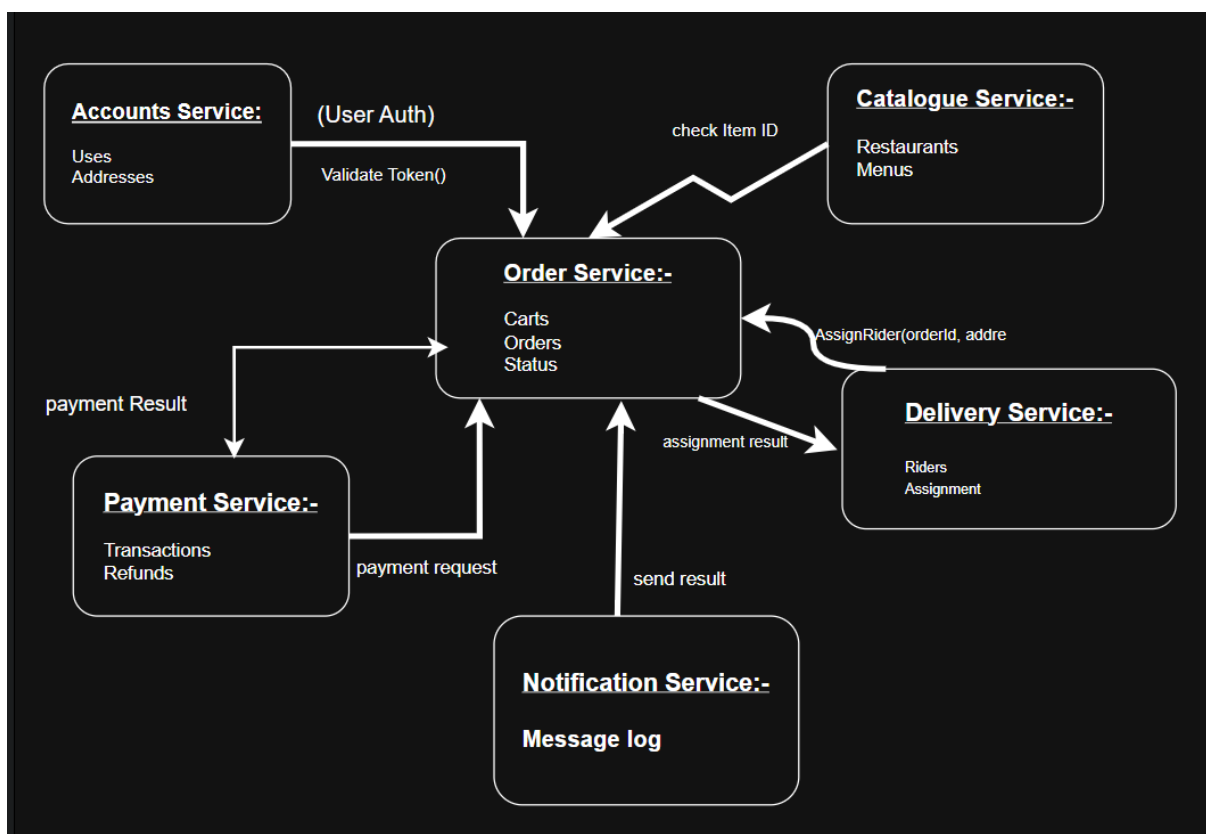


CampusEats(Assignment-2)

1. Capabilities:-

2. Manage profile and addresses:- It allows student to manage their profile and delivery address.
3. Browse restaurants & menus:- It allows student to browse campus restaurants, menus and prices.
4. 0Add to cart & place order:- It allows students to add food items to a cart and place an order.
5. Pay/refund:- handles online payments and refunds.
6. Assign rider & track delivery:- It assign a rider and allows delivery tracking.
7. Send notifications:- it sends notifications when the order status changes.

2. Service design diagram:-



Service Boundaries & team Responsibility:-

Service	Data Owned	Responsible
Identity & Admin Service	Users, roles, vendor accounts, campus administration	Anshu Mala
Catalogue Service	Vendors, menus, food items, prices, availability, pickup locations	Anshu Mala
Order Service	Carts, orders, order items, order status	Mritunjay Maurya
Payment Service	Payment records, payment status, payment attempts	Sanika Jain
Notification Service	Notifications and delivery status	Annu Mishra

3. Service Contracts:-

The Catalogue service own restaurants, and prices. Other services can access this information only through the following published operations.

Catalogue Service:-

Operation	Input	Output	Error
ListRestaurants	Area/filter	Restaurants[]	_____

getMenu	restaurantId	menuItems + prices	Not found
checkItem	itemId	Available?, price	Not found

checkItem Contract

Operation: checkItem(itemId)

Input:

- itemId

Output:

- available?
- price

Error:

- not found

The internal database tables and implementation of the Catalogue service are hidden from other services. Other services depend only on these published operations.

Order Service:-

Operation	Input	Output	Error
addToCart	studentId		Item unavailable
placeOrder	studentId. Cart, addressId	orderId, status, total	Empty cart, bad address
getOrder	orderId	Order+status	Not found

cancelOrder	orderId	Status: cancelled	Too late to cancel
-------------	---------	----------------------	-----------------------

4. Operation:-

placeOrder

Input

- studentId
- items: [{itemId, qty}]
- deliveryAddressId
- paymentMethodId

Output

- orderId
- status: 'placed'
- total
- estimatedMinutes

Errors

- item unavailable
- payment declined
- invalid address
- empty cart

Hidden from callers

- orders table

- how IDs are generated
- which rider gets picked
- payment gateway used

The placeOrder operation allows a student to place an order by providing the selected items, delivery address, and payment method. The Orders service returns the order ID, status, total, and estimated delivery time. Internal database structure, ID generation, rider selection, and payment gateway details remain hidden from the caller .

6. Service Validation

Each service of the CampusEats system is checked against the five important properties of a service: reachable, self-contained, has a contract, independent, and loosely coupled. The validation is given below.

Service	Reachable	Self-contained	Has a Contract	Independent	Loosely Coupled	Remarks / Fix
Accounts Service	Yes	Yes	Yes	Yes	Yes	No major issue. It handles user authentication and address-related data independently.
Catalogue Service	Yes	Yes	Yes	Yes	Yes	No major issue. It manages restaurants and menus and provides item-

						checking operations to the Order Service.
Order Service	Yes	Yes	Yes	Yes	Yes	No major issue. It manages carts, orders and order status while communicating with other services through operations.
Payment Service	Yes	Yes	Yes	Yes	Yes	No major issue. It independently handles transactions and refunds and returns the payment result to the Order Service.
Delivery Service	Yes	Yes	Yes	Yes	Yes	No major issue. It manages riders and delivery assignments independently.
Notification Service	Yes	Yes	Yes	Yes	Yes	No major issue. It manages the message log and receives the required information from the Order Service.

Validation Result

All six CampusEats services satisfy the five required properties of a service. Each service has a separate responsibility and owns its own data. The services communicate through defined operations instead of directly accessing each other's internal data. Therefore, the services remain independent and loosely coupled.

No major failure was identified in the current service design, so no significant modification is required.

